

Contest Platform with Live Leaderboard

Database Implementation & Core Functionalities (MongoDB + Redis)

Team	Upayan Mazumder (24BDS0367), Aditya Rawat (24BCE2992), Shloak Sinha (24BDS0378)
Stack	FastAPI + PyMongo (Python), MongoDB 7 (replica set, for transactions), Redis 7

1. Database Implementation

1.1 Collections

Collection	Key fields
users	name, email (unique), passwordHash, role, totalScore, teamIds[]
teams	name, memberIds[], totalScore
contests	title, startTime, endTime, status, createdBy, problemIds[], participants[]
problems	contestId, type (mcq/coding/subjective), difficulty, points, + type-specific fields
submissions	contestId, problemId, submittedBy{refType,refId}, status, score

1.2 Relationships

- Contest → Problems (`problems.contestId`)
- User/Team → Submission (`submissions.submittedBy`)
- Submission → Problem (`submissions.problemId`)
- Team → Users (`teams.memberIds`), with a back-reference `users.teamIds`
- Contest → Users/Teams (`contests.participants` , embedded ref array)

Each collection has a `$jsonSchema` validator. `problems` uses a `oneOf` clause so mcq/coding/subjective documents enforce different required fields at the database level.

1.3 Sample data (minimum 50 required)

Collection	Records
users	15
teams	5
contests	5
problems	15
submissions	48
Total	88 (minimum required: 50)

Data generated by `scripts/seed.py`, not hand-typed. References checked after seeding — every foreign key resolves to a real document (0 broken references).

```
$ python scripts/seed.py
Seed complete. Record counts:
  users: 15   teams: 5   contests: 5   problems: 15   submissions: 48
TOTAL: 88
```

2. CRUD Operations

Collection	Create	Read	Update	Delete
------------	--------	------	--------	--------

users	POST /users	GET /users, /users/{id}	PATCH /users/{id}	DELETE /users/{id}
teams	POST /teams	GET /teams, /teams/{id}	PATCH /teams/{id}, add/remove member	DELETE /teams/{id}
contests	POST /contests	GET /contests, /contests/{id}, /leaderboard	PATCH /contests/{id}, /status	DELETE /contests/{id}
problems	POST /problems	GET /problems?contestId=, /problems/{id}	PATCH /problems/{id}	DELETE /problems/{id}
submissions	POST /submissions	GET /submissions?..., /submissions/{id}	PATCH /submissions/{id}/status	DELETE /submissions/{id}

Evidence — tested against a running server (curl):

```
$ curl -X POST /users -d '{"name":"Doc Evidence User","email":"doc.evidence2@rankstack.io","password":"secret123"}'
{"id":"6ab3f9a5...", "name":"Doc Evidence User", ...} # CREATE

$ curl /users/6ab3f9a5...
{"id":"6ab3f9a5...", "name":"Doc Evidence User", ...} # READ

$ curl -X PATCH /users/6ab3f9a5... -d '{"name":"Doc Evidence User (Updated)}'
{"id":"6ab3f9a5...", "name":"Doc Evidence User (Updated)", ...} # UPDATE

$ curl -X DELETE /users/6ab3f9a5... -> 204
$ curl /users/6ab3f9a5... -> 404 (confirmed deleted)
```

3. Advanced NoSQL Features

3.1 Aggregation pipelines (5)

#	Pipeline	Result (sample)
1	Leaderboard per contest — group by submitter, sum score, sort desc	{totalScore: 40, refId: ..., refType: 'user'}
2	Avg score per problem, grouped by difficulty	{difficulty: 'hard', avgScorePerProblem: 18.17}
3	Submission count by status, per contest	{status: 'correct', count: 8}
4	Distinct problems solved (correct) per user	{name: 'Angie Henderson', distinctProblemsSolved: 4}
5	Team performance — \$lookup teams → submissions via memberIds	{name: 'Team Lavender Trial', totalScore: 195.0}

```
# pipelines[4] — team performance ($lookup excerpt)
{"$lookup": {"from": "submissions", "let": {"memberIds": "$memberIds"},
  "pipeline": [{"match": {"$expr": {"$and": [
    {"$eq": [{"submittedBy.refType", "user"}]},
    {"$in": [{"submittedBy.refId", "$memberIds"]}]}]}],
  "as": "memberSubmissions"}
```

3.2 Indexing

Collection	Index	Keys
users	uniq_email	{email: 1} unique
problems	by_contest	{contestId: 1}
submissions	by_contest_and_submitter (compound)	{contestId: 1, submittedBy.refId: 1}

submissions	by_problem	{problemId: 1}
teams	by_member	{memberIds: 1}
contests	by_status	{status: 1}

```
$ db.problems.find({contestId: ObjectId("...")}).explain()
Winning plan: FETCH -> IXSCAN, Index used: by_contest (not a collection scan)
```

3.3 Transactions (multi-document, on a replica set)

- Create submission + increment `problems.attemptCount` atomically
- Update submission status/score + update submitter's `totalScore` atomically
- Add/remove team member + update `users.teamIds` back-reference atomically

```
with client.start_session() as session:
    with session.start_transaction():
        db["submissions"].insert_one(doc, session=session)
        db["problems"].update_one({"_id": problem_id},
                                   {"$inc": {"attemptCount": 1}}, session=session)
```

Evidence: `problem.attemptCount` before=7 after=8 | `user.totalScore` before=90 after=100

3.4 Redis (leaderboard / sessions / rate limiting)

Key	Type	Used for
leaderboard:<contestId>	Sorted Set	live rank, updated on every score change
session:<sessionId>	String + TTL	login sessions
rate_limit:<userId>	Counter + TTL	max 10 submissions / 60s

\$ 12 rapid POST /submissions from one user -> 201 x10, then 429 x2 (rate limit working)

4. Current Implementation Status

Criterion	Marks	Status
Database implementation (collections, relationships, sample data)	2	Done
CRUD operations completed	3	Done
Advanced NoSQL features (aggregation, indexing, transactions)	5	Done

5. Repository

Git repository at the project workspace ([rankstack/](#)), 15 commits.

```
app/          FastAPI app, models, db config, aggregations, redis, transactions
scripts/      init_db.py, seed.py, demo.py
docker-compose.yml, Makefile, README.md
```

Run: `make up -> make init-db -> make seed -> make demo / make api`